

Lista Avaliativa 4 - Respostas

1)

A memória virtual é uma técnica que permite que programas utilizem mais memória do que a RAM física disponível. O sistema operacional cria uma ilusão de que cada processo possui um espaço de endereçamento próprio e contínuo, independente da quantidade real de memória.

A paginação funciona dividindo o espaço de endereçamento virtual em blocos de tamanho fixo chamados páginas, tipicamente de 4KB. A memória física também é dividida em blocos do mesmo tamanho, chamados molduras de página ou frames. Quando um programa executa, nem todas suas páginas precisam estar na RAM simultaneamente. O sistema operacional mantém as páginas mais usadas na memória física e armazena as demais no disco. Quando o programa tenta acessar uma página que não está na RAM, ocorre uma falta de página (page fault), e o sistema operacional carrega a página necessária do disco para a memória, possivelmente removendo outra página para liberar espaço.

2)

A tradução ocorre através da tabela de páginas. Um endereço virtual é dividido em duas partes: o número da página virtual e o deslocamento dentro da página. Por exemplo, considere um endereço virtual de 32 bits e páginas de 4KB (12 bits para deslocamento). Um endereço virtual seria dividido em 20 bits para o número da página e 12 bits para o deslocamento.

Suponha o endereço virtual 0x00403004. Os 20 bits superiores (0x00403) identificam a página virtual, e os 12 bits inferiores (0x004) são o deslocamento. A MMU consulta a tabela de páginas usando o número da página virtual como índice e obtém o número da moldura física correspondente. Se a moldura for 0x00A1C, o endereço físico resultante é formado concatenando a moldura com o deslocamento: 0x00A1C004.

Sim, é perfeitamente possível que o endereço virtual tenha mais bits que o físico. Por exemplo, sistemas de 64 bits podem ter endereços virtuais de 48 bits enquanto a RAM física suporta apenas 36 bits de endereçamento. Isso funciona porque a memória virtual não precisa estar toda mapeada na RAM simultaneamente. As páginas não utilizadas ficam no disco ou simplesmente não são alocadas.

A MMU é o hardware responsável por realizar essa tradução automaticamente a cada acesso à memória. Ela intercepta os endereços virtuais gerados pela CPU, consulta as tabelas de páginas e produz os endereços físicos correspondentes. A MMU também verifica bits de proteção nas entradas da tabela de páginas e gera exceções quando ocorrem violações de acesso ou faltas de página.

3)

A tabela de páginas é a estrutura de dados que armazena o mapeamento entre páginas virtuais e quadros de página físicos. Para cada página no espaço virtual do processo, existe uma entrada correspondente na tabela de páginas. Uma entrada típica contém, no mínimo, as seguintes informações:

- **Número do Quadro de Página:** O campo mais importante, que aponta para o local na memória física onde a página está.
- **Bit de Presente/Ausente:** Um bit que indica se a página está atualmente na memória (1) ou no disco (0). Se o bit for 0 e a página for acessada, a MMU gera uma falta de página.

- **Bits de Proteção:** Definem as permissões de acesso à página (ex: leitura, escrita, execução). A MMU verifica esses bits a cada acesso para garantir que operações ilegais (como escrever em uma página de código somente leitura) sejam bloqueadas.
- **Bit de Modificação (*Dirty Bit*):** É ativado pelo hardware sempre que ocorre uma escrita na página. Este bit informa ao sistema operacional que o conteúdo da página na memória foi alterado e precisa ser salvo de volta no disco antes que o quadro de página possa ser reutilizado.
- **Bit de Referência (*Accessed Bit*):** É ativado pelo hardware sempre que a página é lida ou escrita. Este bit é fundamental para os algoritmos de substituição de página, pois ajuda o sistema operacional a identificar quais páginas estão sendo usadas ativamente e quais não são.

4)

A TLB é uma memória cache associativa de alta velocidade localizada na MMU. Como consultar a tabela de páginas na RAM a cada acesso à memória seria muito lento, a TLB armazena as traduções mais recentemente usadas.

Quando a CPU gera um endereço virtual, a MMU primeiro verifica se a tradução está na TLB. Se houver um acerto (TLB hit), o endereço físico é obtido imediatamente sem acessar a memória. Se houver uma falta (TLB miss), a MMU precisa percorrer a tabela de páginas na RAM, realizar a tradução e então adicionar essa entrada na TLB para acessos futuros.

A TLB explora a localidade espacial e temporal dos acessos à memória. Programas tendem a acessar repetidamente as mesmas páginas ou páginas próximas, resultando em altas taxas de acerto na TLB. Isso torna a tradução de endereços praticamente transparente em termos de desempenho. Quando ocorre troca de contexto entre processos, a TLB geralmente precisa ser invalidada, pois as traduções de um processo não se aplicam a outro.

5)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <semaphore.h>
#include <fcntl.h>
#include <time.h>

#define TAMANHO_BUFFER 10
// Intervalo de tempo em segundos para o produtor gerar um item
#define PRODUTOR_DORMIR_TEMPO 1
// Intervalo de tempo em segundos para o consumidor consumir um item
#define CONSUMIDOR_DORMIR_TEMPO 2

typedef struct
{
    char buffer[TAMANHO_BUFFER]; // O buffer circular compartilhado
    int in;                       // Índice onde o produtor insere o próximo
    item
```

```

    int out;                // Índice de onde o consumidor retira o
    próximo item

    sem_t mutex;
    sem_t empty;
    sem_t full;
} SharedData;

SharedData *shared_data;

void producer_task()
{
    printf("Processo Produtor iniciado (PID: %d)\n", getpid());

    while (1)
    {
        // Gera uma letra aleatória de 'A' a 'Z'
        char item = 'A' + (rand() % 26);

        sem_wait(&shared_data->empty);
        sem_wait(&shared_data->mutex);

        // --- SEÇÃO CRÍTICA ---
        shared_data->buffer[shared_data->in] = item;
        printf("Produtor produziu: %c na posição %d\n", item, shared_data-
>in);

        shared_data->in = (shared_data->in + 1) % TAMANHO_BUFFER;
        // --- FIM DA SEÇÃO CRÍTICA ---

        sem_post(&shared_data->mutex);
        sem_post(&shared_data->full);

        sleep(PRODUTOR_DORMIR_TEMPO);
    }
}

void consumer_task()
{
    printf("Processo Consumidor iniciado (PID: %d)\n", getpid());

    while (1)
    {
        sem_wait(&shared_data->full);
        sem_wait(&shared_data->mutex);

        // --- SEÇÃO CRÍTICA ---
        char item = shared_data->buffer[shared_data->out];
        printf("Consumidor consumiu: %c da posição %d\n", item,
shared_data->out);

        shared_data->out = (shared_data->out + 1) % TAMANHO_BUFFER;
        // --- FIM DA SEÇÃO CRÍTICA ---
    }
}

```

```

        sem_post(&shared_data->mutex);
        sem_post(&shared_data->empty);

        sleep(CONSUMIDOR_DORMIR_TEMPO);
    }
}

int main()
{
    srand(time(NULL));

    shared_data = mmap(NULL, sizeof(SharedData), PROT_READ | PROT_WRITE,
MAP_SHARED | MAP_ANONYMOUS, -1, 0);

    if (shared_data == MAP_FAILED)
    {
        perror("mmap falhou");
        exit(1);
    }

    shared_data->in = 0;
    shared_data->out = 0;

    sem_init(&shared_data->mutex, 1, 1);
    sem_init(&shared_data->empty, 1, TAMANHO_BUFFER);
    sem_init(&shared_data->full, 1, 0);

    // Cria o processo produtor
    pid_t producer_pid = fork();
    if (producer_pid < 0)
    {
        perror("fork do produtor falhou");
        exit(1);
    }
    else if (producer_pid == 0)
    {
        producer_task();
        exit(0);
    }

    // Cria o processo consumidor
    pid_t consumer_pid = fork();
    if (consumer_pid < 0)
    {
        perror("fork do consumidor falhou");
        exit(1);
    }
    else if (consumer_pid == 0)
    {
        consumer_task();
        exit(0);
    }

    printf("Processo Pai (PID: %d) criou Produtor (PID: %d) e Consumidor

```

```

(PID: %d)\n", getpid(), producer_pid, consumer_pid);
    printf("Pai esperando os filhos terminarem (neste caso, nunca, pois
eles têm loops infinitos).\n");
    printf("Use Ctrl+C para encerrar o programa.\n");

    // Como os filhos estão em um loop infinito, o pai ficará bloqueado
aqui para sempre.
    // Em um programa real, haveria uma condição de parada.
    wait(NULL);
    wait(NULL);

    // 5. LIMPEZA DOS RECURSOS (nunca vai chegar por causa do loop
infinito)
    printf("Limpando recursos...\n");
    sem_destroy(&shared_data->mutex);
    sem_destroy(&shared_data->empty);
    sem_destroy(&shared_data->full);
    munmap(shared_data, sizeof(SharedData));

    return 0;
}

```

6)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <semaphore.h>
#include <pthread.h>

#define MAX_MSGS 100
#define MAX_MSG_LEN 256
#define MAX_USER_LEN 32
#define SHM_NAME "/chat_shm"
#define SEM_NAME "/chat_sem"

typedef struct
{
    char usuario[MAX_USER_LEN];
    char texto[MAX_MSG_LEN];
    int id;
} Mensagem;

typedef struct
{
    Mensagem mensagens[MAX_MSGS];
    int total_msgs;
}

```

```

} ChatMemoria;

ChatMemoria *chat_mem = NULL;
sem_t *sem = NULL;
char meu_usuario[MAX_USER_LEN];
int ultima_msg_lida = 0;
int rodando = 1;

void *monitorar_mensagens(void *arg)
{
    while (rodando)
    {
        sem_wait(sem);

        if (chat_mem->total_msgs > ultima_msg_lida)
        {
            for (int i = ultima_msg_lida; i < chat_mem->total_msgs; i++)
            {
                int idx = i % MAX_MSGS;
                Mensagem *msg = &chat_mem->mensagens[idx];

                printf("\n[%s]: %s\n> ", msg->usuario, msg->texto);
                fflush(stdout);
            }

            ultima_msg_lida = chat_mem->total_msgs;
        }

        sem_post(sem);
        usleep(100000);
    }
    return NULL;
}

void enviar_mensagem(const char *texto)
{
    sem_wait(sem);

    int idx = chat_mem->total_msgs % MAX_MSGS;

    strncpy(chat_mem->mensagens[idx].usuario, meu_usuario, MAX_USER_LEN -
1);
    strncpy(chat_mem->mensagens[idx].texto, texto, MAX_MSG_LEN - 1);
    chat_mem->mensagens[idx].id = chat_mem->total_msgs;

    chat_mem->total_msgs++;

    sem_post(sem);
}

void inicializar_chat()
{
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1)

```

```

{
    perror("Erro ao criar memória compartilhada");
    exit(1);
}

if (ftruncate(shm_fd, sizeof(ChatMemoria)) == -1)
{
    perror("Erro ao dimensionar memória compartilhada");
    exit(1);
}

chat_mem = mmap(NULL, sizeof(ChatMemoria),
                PROT_READ | PROT_WRITE,
                MAP_SHARED,
                shm_fd, 0);

if (chat_mem == MAP_FAILED)
{
    perror("Erro ao mapear memória compartilhada");
    exit(1);
}

sem = sem_open(SEM_NAME, O_CREAT, 0666, 1);
if (sem == SEM_FAILED)
{
    perror("Erro ao criar semáforo");
    exit(1);
}

close(shm_fd);
}

void limpar_chat()
{
    if (chat_mem != NULL)
    {
        munmap(chat_mem, sizeof(ChatMemoria));
    }

    if (sem != NULL)
    {
        sem_close(sem);
    }
}

int main()
{
    char buffer[MAX_MSG_LEN];
    pthread_t thread_monitor;

    printf("||\n");
    printf("||  CHAT COM MEMÓRIA COMPARTILHADA  ||\n");
    printf("||\n\n");

```

```

printf("Digite seu nome de usuário: ");
if (fgets(meus_usuario, MAX_USER_LEN, stdin) == NULL)
{
    fprintf(stderr, "Erro ao ler nome de usuário\n");
    return 1;
}
meus_usuario[strcspn(meus_usuario, "\n")] = 0;

if (strlen(meus_usuario) == 0)
{
    fprintf(stderr, "Nome de usuário não pode ser vazio\n");
    return 1;
}

inicializar_chat();

sem_wait(sem);
ultima_msg_lida = chat_mem->total_msgs;
sem_post(sem);

printf("\n✓ Chat iniciado como '%s'\n", meus_usuario);
printf("✓ Digite suas mensagens (ou 'sair' para encerrar)\n\n");

if (pthread_create(&thread_monitor, NULL, monitorar_mensagens, NULL) !=
0)
{
    perror("Erro ao criar thread");
    limpar_chat();
    return 1;
}

while (1)
{
    printf("> ");
    fflush(stdout);

    if (fgets(buffer, MAX_MSG_LEN, stdin) == NULL)
    {
        break;
    }

    buffer[strcspn(buffer, "\n")] = 0;

    if (strcmp(buffer, "sair") == 0)
    {
        break;
    }

    if (strlen(buffer) > 0)
    {
        enviar_mensagem(buffer);
    }
}

```



```
printf("\nEncerrando chat...\n");  
rodando = 0;  
pthread_join(thread_monitor, NULL);  
limpar_chat();  
  
printf("Chat encerrado!\n");  
  
return 0;  
}
```